

Theoretical Assignment – Question 3

PySpark Joins, Ambiguous References, Column Selection & File Reading

1. What is a Join in PySpark?

A **join** combines rows from two DataFrames based on a related column or condition. In a Retail/Sales system, an orders DataFrame may contain customer_id while a customers DataFrame contains customer details. The common customer_id connects the datasets.

```
orders.customer_id == customers.customer_id
      \           /
       \   match  /
        \  JOIN  /
```

The main idea is: **Join = connect related rows using a common key or condition.**

2. Seven Core Types of Joins in PySpark

2.1 Inner Join

Returns only rows that match in both DataFrames.

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "inner"
)
```

With orders C101, C102, C103 and customers C101, C102, C104, the result contains C101 and C102 only.

INNER → matching rows only.

2.2 Left Outer Join

Keeps every row from the left DataFrame. Matching right-side data is added; unmatched right-side columns become NULL.

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "left"
)
```

C103 remains because it exists in orders, even though it has no matching customer.

LEFT → all left rows + matching right data.

2.3 Right Outer Join

Keeps every row from the right DataFrame. Unmatched left-side columns become NULL.

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "right"
)
```

C104 remains because it exists in customers even though there is no corresponding order.

RIGHT → all right rows + matching left data.

2.4 Full Outer Join

Keeps all rows from both DataFrames. Matching rows are combined; unmatched rows contain NULLs for the missing side.

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "full"
)
```

The result contains C101, C102, C103 and C104.

FULL → everything from both sides.

2.5 Cross Join

Combines every row from the left with every row from the right. There is no matching condition.

```
result = orders.crossJoin(customers)
```

If orders has 3 rows and customers has 3 rows, the result has $3 \times 3 = 9$ combinations. Large inputs can therefore create a very large result.

CROSS → every left row × every right row.

2.6 Left Semi Join

Returns rows from the left only when a matching row exists in the right. Right-side columns are not returned.

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "left_semi"
)
```

C101 and C102 are kept because they exist in customers. C103 is removed.

LEFT SEMI → matching rows from LEFT only.

2.7 Left Anti Join

Returns rows from the left only when no matching row exists in the right.

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "left_anti"
)
```

C103 is kept because there is no C103 in customers.

LEFT ANTI → non-matching rows from LEFT only.

Join Types at a Glance

Join	What is kept?
inner	Only matching rows
left	All left rows + matching right data
right	All right rows + matching left data
full	All rows from both sides
cross	Every possible combination
left_semi	Left rows that have a match
left_anti	Left rows that do not have a match

3. Ambiguous Reference Error in PySpark

An **Ambiguous Reference** error occurs when Spark sees multiple columns with the same name and a later operation refers to that name without identifying which column is intended. This commonly happens after joins.

```
joined_df = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "inner"
)

joined_df.select("customer_id")
```

After the join, Spark may have both `orders.customer_id` and `customers.customer_id`. So `select("customer_id")` is ambiguous: Spark cannot determine which one you mean.

Fix: Use DataFrame aliases

```
o = orders.alias("o")
c = customers.alias("c")

joined_df = o.join(
    c,
    o.customer_id == c.customer_id,
    "inner"
)

joined_df.select(
    "o.customer_id",
    "o.order_id",
    "c.customer_name"
).show()
```

Alternative: Rename a duplicate column

```
customers = customers.withColumnRenamed(
    "customer_id",
    "cust_id"
)

result = orders.join(
    customers,
    orders.customer_id == customers.cust_id
)
```

Remember: Ambiguous reference = Spark has more than one possible column with the same name. Use aliases or distinct column names.

4. Difference Between select() and selectExpr()

select() is used to choose columns and construct PySpark Column expressions. **selectExpr()** lets you write SQL-style expressions as strings.

Using select()

```
df.select(
    "product_name",
    "quantity",
    "unit_price",
    (df.quantity * df.unit_price).alias("gross_amount")
)
```

Using selectExpr()

```
df.selectExpr(
    "product_name",
    "quantity",
    "unit_price",
    "quantity * unit_price AS gross_amount"
)
```

Both can create gross_amount. The difference is how the transformation is expressed: select() uses PySpark Column expressions, while selectExpr() accepts SQL-style expression strings.

Another selectExpr() example

```
df.selectExpr(
    "quantity * 2 AS double_quantity",
    "UPPER(customer_name) AS customer_name"
)
```

Memory trick: **select()** → PySpark column expressions. **selectExpr()** → SQL-style expressions as strings.

5. Fundamental Difference Between left_semi and left_anti

Both return columns from the **left DataFrame**. The difference is whether a matching row exists in the right DataFrame.

```
LEFT SEMI
Left row → Does match exist on RIGHT?
        YES → KEEP
        NO  → REMOVE

LEFT ANTI
```

Left row → Does match exist on RIGHT?
YES → REMOVE
NO → KEEP

left_semi

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "left_semi"
)
```

C101 and C102 are returned because matching customer IDs exist.

left_anti

```
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,
    "left_anti"
)
```

C103 is returned because no matching customer ID exists.

SEMI = matching left records.
ANTI = non-matching left records.

6. Reading Different File Formats in PySpark

PySpark provides DataFrameReader methods for common formats. The general pattern is **spark.read** → **DataFrame**.

6.1 Text / CSV

```
df = spark.read.csv(
    "sales_data.csv",
    header=True,
    schema=sales_schema
)
```

header=True tells Spark that the first row contains column names. schema=sales_schema applies the explicit Retail/Sales schema.

6.2 Avro

```
df = spark.read.avro("sales_data.avro")

# Alternative
df = spark.read.format("avro").load("sales_data.avro")
```

6.3 Parquet

```
df = spark.read.parquet("sales_data.parquet")
```

6.4 ORC

```
df = spark.read.orc("sales_data.orc")
```

Reading Syntax Comparison

Format	PySpark reading syntax
Text/CSV	spark.read.csv("sales_data.csv", header=True, schema=sales_schema)
Avro	spark.read.avro("sales_data.avro")
Parquet	spark.read.parquet("sales_data.parquet")
ORC	spark.read.orc("sales_data.orc")

Overall PySpark Data Processing Flow

```
File
↓
spark.read.<format>()
↓
DataFrame
```

↓
select / selectExpr / withColumn / join
↓
Transformation / Analysis
↓
spark.write
↓
Output

Important Points to Remember

- **Join** connects related rows from two DataFrames using a key/condition.
- **Inner** = matching only; **left** = all left; **right** = all right; **full** = all rows from both.
- **Cross join** creates every possible left-right combination.
- **left_semi** = left rows with a match; **left_anti** = left rows without a match.
- **Ambiguous reference** means Spark cannot determine which same-named column is intended.
- **select()** uses PySpark Column expressions; **selectExpr()** uses SQL-style expression strings.
- File reading commonly follows **spark.read.<format>()**.